FINAL YEAR TECHNICAL SEMINAR REPORT

ON

# Semantic Caching for RAG and LLM Pipelines
## Correctness and Cost Trade-offs Under Controlled Query Variation

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology
Rajasthan Technical University




**Submitted To**

Ms. Khushboo Dadhich

Associate Professor

Department of Computer Engineering

**Submitted By**

Garv Jain

PIET23CR018

4th Year, 7th Sem

DEPARTMENT OF COMPUTER ENGINEERING POORNIMA INSTITUTE OF ENGINEERING & TECHNOLOGY, JAIPUR
ISI - 02, RIICO INSTITUTIONAL AREA, SITAPURA, JAIPUR
(Academic Year 2026-27)

FINAL YEAR TECHNICAL SEMINAR REPORT

# Semantic Caching for RAG and LLM Pipelines

## Correctness and Cost Trade-offs Under Controlled Query Variation

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology
Rajasthan Technical University




Submitted by
Garv Jain ( PIET23CR018)

DEPARTMENT OF COMPUTER ENGINEERING POORNIMA INSTITUTE OF ENGINEERING & TECHNOLOGY, JAIPUR
ISI - 02, RIICO INSTITUTIONAL AREA, SITAPURA, JAIPUR
(Academic Year 2026-27)

# CERTIFICATE

This is to certify that the Final Year Technical Seminar Report entitled " **Semantic Caching for RAG and LLM Pipelines: Correctness and Cost Trade-offs Under Controlled Query Variation** " has been submitted by **Garv Jain (PIET23CR018**) for partial fulfilment of the Degree of Bachelor of Technology of Rajasthan Technical University. It is found satisfactory and approved for submission.

Date: 28 October 2026

Dr. Anil Kumar
Head,
Dept. of Comuter Engineering
PIET, Jaipur

Dr. Dinesh Goyal
Director
PIET, Jaipur

## DECLARATION

I hereby declare that the seminar report entitled " **Semantic Caching for RAG and LLM Pipelines: Correctness and Cost Trade-offs Under Controlled Query Variation**" was carried out and written by me under the guidance of **Ms. Khushboo Dadhich**, Assistant Professor, Department of Computer Engineering, Poornima Institute of Engineering & Technology, Jaipur. This work has not previously formed the basis for the award of any degree or diploma or certificate, nor has it been submitted elsewhere for the award of any degree or diploma.

Place: Jaipur
Date: 28 October, 2026

**Garv Jain**
**PIET23CR018**

## Acknowledgment

This seminar would not have been possible without the help and support of many people, and I would like to thank all of them. First of all, I am thankful to Mr. Shashikant Singhi, Chairman, PGC, and Dr. Dinesh Goyal, Director, PIET, for giving me the platform and the facilities to carry out this technical seminar.

I am also very grateful to Dr. Anil Kumar, Head of the Department of Computer Engineering, for his kind support.

I would like to thank Ms. Jayshree Surolia, Assistant Professor, and Dr. Ashwini Kumar, Associate Professor, who coordinated the Final Year Technical Seminar, for guiding and motivating me at every step and helping me complete the seminar on time.

I am especially thankful to my seminar guide, Ms. Khushboo Dhadhich, Assistant Professor, Department of Computer Engineering, for his support, for sharing his knowledge of the subject, and for his helpful suggestions while I was writing the review paper and this report.

I would also like to thank all the faculty members of the Department of Computer Engineering for their guidance and moral support, and my friends for their suggestions, which helped me improve this work.

Submitted By:

**Garv Jain**

**PIET23CR018**

## Abstract

Semantic caches reuse a stored answer when a new query is sufficiently similar, but similarity alone does not ensure that the answer fits. This project evaluates four matchers on a controlled workload of 700 queries derived from 100 NQ-Open anchors: 400 paraphrases, 200 hard negatives across four perturbation types, and 100 unrelated queries. At a common 40% cache-hit rate, the cross-encoder had the lowest observed false-hit rate against construction labels (16.4%, 46/280), followed by BGE cosine (26.8%), MiniLM cosine (33.6%), and lexical matching (36.4%). At this matched rate, the cross-encoder missed 41.5% of paraphrases. At $\tau$=0.75, negation hit rates were 24% for the cross-encoder and 98% for each bi-encoder; this threshold-specific result is separate from the matched-rate comparison. The response judge failed its validation gates, so construction-label outcomes are primary and judge-based false-hit rate is secondary. Construction-label outcomes measure retrieval against designed query–anchor relationships; they do not independently establish the factual correctness of every reused response. The results describe one controlled workload, and Wilson intervals overlap for some matcher comparisons.



**Subject Descriptors:**

- Semantic caching
- Retrieval-augmented generation (RAG)
- Large language models (LLMs)
- Paraphrase detection
- Hard-negative evaluation
- False-hit rate, False-miss rate
- Query matching.

**Keywords**: semantic caching; retrieval-augmented generation; false-hit rate; hard negatives; paraphrase; threshold; language-model judge

**Implementation Software and Hardware**:

Python, Ollama, Sentence Transformer, FAISS, Hugging Face Datasets, `difflib`, MiniLM, BGE, cross-encoder, Gemma, Llama 3.2, and Phi-4-mini models, AMD Ryzen 5 processor with integrated graphics, CPU-only inference, no discrete GPU.

## Table of Contents

List of Tables ..... 9
List of Figures ..... 10
Abbreviations ..... 11
Symbols and Nomenclature ..... 12
Chapter 1 Introduction ..... 13
1.1 Why this problem ..... 13
1.2 Areas of application ..... 13
1.3 History and development ..... 14
1.4 Description and significance of the topic ..... 15
1.5 Objectives and research questions ..... 16
1.6 Scope and organisation of the report ..... 17
Chapter 2 Technology Specification and Literature Review ..... 18
2.1 Language and runtime ..... 18
2.2 Tools and libraries ..... 18
2.3 Models ..... 18
2.4 Techniques ..... 19
2.5 Literature review ..... 21
2.6 Synthesis and the research gap ..... 23
Chapter 3 Topic Description and Work Performed ..... 24
3.1 Problem setting and overview ..... 24
3.2 Dataset ..... 24
3.3 Preprocessing ..... 25
3.4 Corpus construction ..... 27
3.5 Label verification ..... 28
3.6 Matching methods: the process model ..... 29
3.7 The judge ..... 30
3.8 Metrics ..... 31
3.9 Experimental setup and protocol ..... 33
3.10 Analysis and results ..... 35
Chapter 4 Limitations ..... 44
4.1 The variants come from one generator ..... 44
4.2 The label audit is small and its raters are not fully independent ..... 44
4.3 The size of the corpus and dependence between queries ..... 44
4.4 The kind of question ..... 44
4.5 A static cache with no eviction ..... 44

4.6 An exact index ..... 44
4.7 Local, processor-only cost ..... 44
4.8 The judge ..... 45
4.9 Scores are not on a common scale ..... 45
4.10 A limited set of matching methods ..... 45
4.11 Single turn and English only . **Error! Bookmark not defined.**
4.12 The matched point is chosen on the evaluation data ..... 45
Chapter 4 Future Scope ..... 46
5.1 A larger corpus and a broader audit ..... 46
5.2 Deployment and domain transfer ..... 46
5.3 Matcher and judge policies ..... 46
Chapter 5 Conclusion ..... 47
Appendix A. Judge Prompt ..... **Error! Bookmark not defined.**
Judge prompt ..... 49
References ..... 50

## List of Tables

TABLE 1.1. Dated milestones from the sources we reviewed..... 14

TABLE 2.1. Tools and the role each plays..... 18

TABLE 2.2. Models and the role each plays..... 18

TABLE 2.3. The reviewed work along four dimensions..... 22

TABLE 3.1. Kinds of evaluation query..... 25

TABLE 3.2. Hard-negative types, with illustrative pairs written for this report (not taken from the corpus)..... 27

TABLE 3.3. The four matching configurations..... 29

TABLE 3.4. Settings used in the experiments..... 33

TABLE 3.5. Composition of the evaluation set and mean covariates..... 35

TABLE 3.6. The four configurations at exactly 40% model-call savings. ..... 37

TABLE 3.7. Hard-negative hit rate and secondary JFH by kind of query at exactly 40% model-call savings. ..... 38

TABLE 3.8. Paraphrase false-miss rate by level and by measured wording distance, at exactly 40% model-call savings. ..... 39

TABLE 3.9. Phi-4-mini disagreement with the audit consensus, by stratum. .. 41

TABLE 3.10. Judge-label disagreement among hits at exactly 40% model-call savings..... 41

TABLE 3.11. Direction of the disagreements at exactly 40% model-call savings, derived from the counts in Tables 3.6 and 3.10..... 41

TABLE 3.12. Published findings on judge agreement, error control and negation, and what this work shows..... 42

## List of Figures

Fig. 3.1. End-to-end pipeline. ..... 24

Fig. 3.2. The structure of the corpus around one anchor. ..... 28

Fig. 3.3. The replay design. ..... 34

Fig. 3.4. Judged false-hit rate among hits over each configuration's threshold grid (secondary diagnostic).** The denominator is the number of hits for that configuration and threshold. ..... 36

Fig. 3.5. Construction-label FHR against model-call savings at the tested thresholds; markers show an exact matched 40% hit-share point. ..... 36

Fig. 3.6. Share of the queries of each hard-negative type that are served from the cache, at exactly 40% model-call savings.** Bars are 95% Wilson intervals. Every hard-negative hit is a label false hit. ..... 38

Fig. 3.7. Paraphrase false-miss rate by paraphrase level, token Jaccard bin and edit-distance bin, at exactly 40% model-call savings.** Bars are 95% Wilson intervals. ..... 40

Fig. 3.8. Judge–label disagreement by query subtype at the exact matched 40% cache-hit share. ..... 42

## Abbreviations

LLM — large language model

RAG — retrieval-augmented generation

FHR — false-hit rate

FMR — false-miss rate

JFH — judged false-hit rate

NQ-Open — Natural Questions open-domain question-answering subset

AI — Artificial Intelligence

API — Application Programming Interface

BGE — BAAI General Embedding

CE — Cross-Encoder

CPU — Central Processing Unit

E — Entity Substitution

FHR — False-Hit Rate

FMR — False-Miss Rate

GPU — Graphics Processing Unit

IR — Information Retrieval

JFH — Judged False-Hit Rate

LLM — Large Language Model

MTEB — Massive Text Embedding Benchmark

N — Negation

NQ-Open — Natural Questions Open-Domain Question Answering

P1–P4 — Paraphrase Levels 1–4

RAG — Retrieval-Augmented Generation

RQ — Research Question

S — Scope Change

T — Number or Date Change

$\tau$ — Matcher-Specific Similarity Threshold

UI — User Interface

## Symbols and Nomenclature

q — incoming query

a*(q) — nearest anchor selected for query q

s(q) — matcher score assigned to q

$\tau$ — matcher-specific cache-hit threshold

FHR — construction-label false hits divided by cache hits

FMR — paraphrase false misses divided by all paraphrases

JFH — judge-based false hits divided by cache hits

$S_{(C)}$ — fraction of queries served from the cache

$\kappa$ — Cohen's kappa coefficient

$d_e$ — normalized edit distance between two queries

# Chapter 1 Introduction

## 1.1 Why this problem

Language-model calls consume time and, for hosted models, money. Because many requests repeat an earlier question in different words, a semantic cache can reuse a stored answer and avoid generation (Chen et al., 2023; Li et al., 2024). It represents questions as vectors, retrieves the nearest stored question, and serves its response when a matcher score passes a threshold.

Similarity is only a proxy for equivalent meaning. For example, “population of Jaipur” and “population of Rajasthan” are lexically close but require different answers, while a longer paraphrase may look less similar. A false hit can therefore return a confident answer to the wrong question; a false miss merely triggers another model call.

This report measures that trade-off under controlled query variation. Prior work notes that a static threshold does not guarantee a chosen error rate (Schroeder et al., 2026). We evaluate what four stored-score matchers admit at different savings levels and which changes they confuse; we do not propose a new cache or a deployment guarantee.

## 1.2 Areas of application

Semantic caching may help wherever questions recur. The examples below distinguish settings reported in prior work from plausible applications by analogy; only the former are supported by cited evaluations.

1. Chat assistants reuse recurring conversational questions; a false hit can answer the wrong conversational need (Li et al., 2024; Singh et al., 2026).
2. Search queries are short and sensitive to a single changed term, making entity and number confusions consequential (Singh et al., 2026).
3. A cache before a RAG pipeline can skip retrieval and generation, but a false hit may reuse an answer grounded in the wrong documents (Lewis et al., 2020; Jin et al., 2024).
4. Multi-turn systems must account for dialogue history; this study excludes that context (Yan et al., 2025; Yu et al., 2025).
5. On-device assistants can reuse personal answers while keeping data local (Gill et al., 2025a).

6. Medical question answering requires particular care because similar wording can request different advice (Gill et al., 2025b).
7. Question-and-answer sites can use semantic similarity to identify duplicate questions (Gill et al., 2025b).
8. Paid model APIs create a direct incentive to avoid repeat calls (Chen et al., 2023).
9. Customer-support and FAQ systems may reuse common replies, but must distinguish product, plan, and policy details; this is an application by analogy.
10. Enterprise assistants may reuse procedural answers while guarding against near-identical requests for different policies; this is an application by analogy.
11. Education assistants may see repeated questions about deadlines or assignments, where year and assignment identifiers matter; this is an application by analogy.
12. Developer assistants may reuse solutions only when language, library, and version match; this is an application by analogy.
13. Voice assistants may encounter repeated spoken requests, but users have less opportunity to inspect a mistaken answer; this is an application by analogy.

## 1.3 History and development

The timeline in Table 1.1 groups the cited work into phases; this grouping is our synthesis, not a standard classification.

*TABLE 1.1. Dated milestones from the sources we reviewed.*

| When | Work | What it added |
|---|---|---|
| Apr 2019 | PAWS (Zhang et al., 2019) | Paraphrase and non-paraphrase pairs with high word overlap, to test paraphrase models |
| Aug 2019 | Sentence-BERT (Reimers and Gurevych, 2019) | Sentence embeddings that can be compared directly by vector similarity |
| Apr–May 2020 | Contrast sets (Gardner et al., 2020); CheckList (Ribeiro et al., 2020) | Test sets built from small, targeted edits to expose model failures |
| May 2020 | Retrieval-augmented generation (Lewis et al., 2020) | The retrieve-then-generate pattern that caches are later placed in front of |
| May 2023 | FrugalGPT (Chen et al., 2023) | Strategies for cutting the cost of language-model use |
| Nov 2023 | Prompt Cache (Gim et al., 2023) | Reuse of attention states for repeated prompt parts |
| Dec 2023 | GPTCache (Bang, 2023) | An open-source semantic cache that stores model responses |
| Mar 2024 | MeanCache (Gill et al., 2025a) | A user-side cache with a federated-learned similarity model |
| Apr 2024 | RAGCache (Jin et al., 2024) | Caching of the internal states of retrieved documents |

| When | Work | What it added |
|---|---|---|
| Jun 2024 | SCALM (Li et al., 2024) | Cache design based on an analysis of real chat data |
| Nov 2024 | GPT Semantic Cache (Regmi and Pun, 2024) | An embedding-based caching layer for model queries |
| Feb 2025 | vCache (Schroeder et al., 2026) | A cache with a user-defined error-rate guarantee |
| Apr 2025 | Domain-specific embeddings for caching (Gill et al., 2025b) | A fine-tuned embedding model for caching, with synthetic data |
| Jun 2025 | ContextCache (Yan et al., 2025) | Context awareness for multi-turn queries |
| 2025 | SmartCache (Yu et al., 2025) | Context-aware caching with cross-session reuse of attention state (NeurIPS 2025) |
| Feb 2026 | Krites (Singh et al., 2026) | A judge that verifies borderline matches off the serving path |

The work progresses from embeddings and targeted evaluation (Reimers and Gurevych, 2019; Zhang et al., 2019; Gardner et al., 2020; Ribeiro et al., 2020), through cost reduction and cache systems (Gim et al., 2023; Bang, 2023; Li et al., 2024), to context-aware and correctness-oriented policies (Yan et al., 2025; Yu et al., 2025; Schroeder et al., 2026; Singh et al., 2026). This study addresses a narrower question: what a plain threshold admits by query type.

## 1.4 Description and significance of the topic

A query is the incoming question; an anchor is a question with a stored response. A matcher selects an anchor and assigns a score, and threshold $\tau$ determines whether its response is reused (a hit) or generation is triggered (a miss). A true hit retrieves the source anchor for a paraphrase; a false hit retrieves any other query–anchor relation. A false miss is a paraphrase not served. Hard negatives look similar but require a different answer: entity substitution (E), negation (N), number or date change (T), and scope change (S).

The topic matters for three reasons.

False misses cost a model call; false hits can mislead users with an answer to a different question. Threshold selection must therefore consider correctness alongside savings.

Aggregate hit rates obscure which near-misses cause errors. A controlled, typed corpus lets us separate entity, negation, numeric or temporal, and scope changes and compare those outcomes across matchers.

The study therefore reports both the cost trade-off and errors by query type, while distinguishing construction-label correctness from the response judge's acceptability decision.

## 1.5 Objectives and research questions

The objectives of the work are:

14. To build a corpus from real questions in which the relation of each evaluation query to the cache is known by construction, and to check that relation on a stratified audit.
15. To measure hit acceptability, apply the prespecified response-judge gates; if either fails, use construction-label false-hit and hard-negative hit rates as primary measures and report judge outcomes as secondary diagnostics.
16. To compare four matching configurations on the same queries, the same cache and the same judge, under one cost accounting.
17. To make the threshold sweep cheap and exactly repeatable, by computing every expensive step once and replaying the stored results.
18. To report the results by query type, so that a threshold can be understood in terms of what it fails on.

These objectives lead to four research questions.

RQ1. How does the false-hit rate move with the threshold for each type of near-miss, and which types does a threshold fail on first?

RQ2. How often does a judge's verdict on the reused response disagree with the label of the matched pair, and for which types?

RQ3. Can a typed, audited corpus built from real questions show differences between matching methods that aggregate evaluation hides?

RQ4. Under identical queries, cache and judge, how do bi-encoder matching with two embedding models, cross-encoder reranking and lexical matching trade false hits against cost savings?

## 1.6 Scope and organisation of the report

The study uses short, single-turn English factual questions, a static cache, exact nearest-neighbour search, and local processor inference. Savings are reported as model calls avoided, not money. Chapter 4 discusses the implications of these choices.

The report is organised as follows. Chapter 2 describes the technology and related work; Chapter 3 explains the study and reports the results; Chapter 4 presents snapshots of the work; Chapters 5 and 6 discuss limitations and future scope; Chapter 6 concludes.

# Chapter 2 Technology Specification and Literature Review

## 2.1 Language and runtime

The whole pipeline is written in Python. We chose it for a practical reason: every library we need for embeddings, vector search, dataset loading and local model serving has a Python interface, so the pipeline can be one program and not a chain of tools in different languages. The work runs on one consumer machine with an AMD Ryzen 5 processor and integrated graphics. There is no discrete graphics card, so every model, whether it generates text, embeds it or scores it, runs on the processor. This constraint shapes the study. It limits the size of the models we can use and the size of the corpus we can evaluate in a few days of compute, and it means that every latency reported in this work describes processor inference.

## 2.2 Tools and libraries

*TABLE 2.1. Tools and the role each plays.*

| Tool | Role in this work |
|---|---|
| Python | Language of the whole pipeline |
| Ollama | Serves the three generative models locally and exposes them through a simple interface |
| sentence-transformers | Loads the embedding models and the cross-encoder; the library goes with the Sentence-BERT approach (Reimers and Gurevych, 2019) |
| FAISS | Holds the vectors of the cached questions and returns the nearest one (Johnson et al., 2021) |
| Hugging Face datasets | Loads the NQ-Open questions at a pinned revision (Lhoest et al., 2021) |
| difflib (Python standard library) | Computes the character-level sequence ratio used by the lexical matcher |

## 2.3 Models

*TABLE 2.2. Models and the role each plays.*

| Role | Model | Size or dimension | Source |
|---|---|---|---|
| Writes paraphrases and hard negatives | Gemma 3 | 4 billion parameters (the family has 1B, 4B, 12B and 27B) | (Gemma Team, 2025) |
| Writes the stored and fallback responses, at temperature 0 | Llama 3.2 | 3 billion parameters | Llama 3 family (Meta AI, 2024) |
| Judges whether a reused response is acceptable, at temperature 0 | Phi-4-mini | 3.8 billion parameters | (Microsoft, 2025) |

| Role | Model | Size or dimension | Source |
|---|---|---|---|
| Embedding model 1 | all-MiniLM-L6-v2 | 384-dimensional vectors | MiniLM (Wang et al., 2020), through Sentence-BERT (Reimers and Gurevych, 2019) |
| Embedding model 2 | bge-base-en-v1.5 | 768-dimensional vectors | BGE (Xiao et al., 2023) |
| Cross-encoder reranker | cross-encoder/quora-distilroberta-base | Scores a pair of questions | Model trained on duplicate-question pairs (Cross-Encoder, 2026) |

The three generative models come from three different developers, and we did that on purpose. A judge that belongs to the same family as the model whose text it grades tends to favour that text (Panickssery et al., 2024), so we want the judge to be a different family from the one that wrote the paraphrases and from the one that wrote the responses. All three are small enough to run on a processor in reasonable time.

## 2.4 Techniques

This section explains the techniques the pipeline relies on. For each one we give the idea, the formula where there is one, and a small worked example. The examples use the same question pairs that appear in Chapter 1, so that the numbers can be compared.

### 2.4.1 Sentence embeddings and cosine similarity

Sentence-BERT and contrastive methods learn vectors that place semantically related texts near one another (Reimers and Gurevych, 2019; Gao et al., 2021). Dense retrieval uses the same separate-encoding principle (Karpukhin et al., 2020). BGE models are also evaluated on MTEB (Xiao et al., 2023; Muennighoff et al., 2023).

The usual measure of closeness is cosine similarity, the cosine of the angle between two vectors:

$$\cos(u, v) = (u \cdot v) / (\|u\| \, \|v\|)$$

For unit-normalised vectors, the dot product equals cosine similarity; our retrieval implementation uses this property.

### 2.4.2 Exact and approximate nearest-neighbour search

A flat index compares the query with every stored vector and returns the exact nearest neighbour. Approximate graph indexes trade exactness for speed at larger scale (Malkov and

Yashunin, 2020); FAISS supports both approaches (Johnson et al., 2021). With 100 anchors, exact search avoids index approximation as a source of error.

### 2.4.3 Cross-encoder reranking

A bi-encoder encodes each text separately; a cross-encoder scores a query and candidate jointly, allowing token interactions at higher per-pair cost. A common design retrieves a small candidate set first, then reranks it (Nogueira and Cho, 2019). Here, MiniLM supplies five candidates and a duplicate-question cross-encoder (Cross-Encoder, 2026) selects the final match.

### 2.4.4 Lexical sequence matching

The lexical matcher uses Python's SequenceMatcher ratio, 2M/T, where M is the number of characters in matching blocks and T is the combined string length. It is a surface-form baseline without learned representations.

### 2.4.5 Token-set overlap and near-duplicate removal

Token Jaccard similarity is the size of the intersection of two lowercase token sets divided by the size of their union.

Jaccard similarity supports near-duplicate filtering and ensures unrelated queries have low overlap with anchors. A prefix filter limits candidate comparisons, after which each candidate pair is checked exactly (Chaudhuri et al., 2006); deduplication matters because repeated text can contaminate evaluation (Lee et al., 2022).

### 2.4.6 Normalised edit distance

Normalised edit distance divides the minimum character insertions, deletions, or substitutions needed to transform one string into another by string length. We record it alongside token Jaccard to measure wording distance from the source anchor.

### 2.4.7 Language-model judging

Language-model judges can agree with human preference labels, but published work also identifies position, verbosity, and self-preference biases (Zheng et al., 2023; Liu et al., 2023; Panickssery et al., 2024). Those results concern stronger models; the small local judge used here is validated against stored anchors and an audit before its rate can be primary.

### 2.4.8 Agreement between judge and an independent rater: Cohen's κ

Cohen's κ adjusts observed agreement $p_o$ for agreement expected by chance $p_e$ (Cohen, 1960):

$$\kappa = (p_o - p_e) / (1 - p_e)$$

The prespecified gate requires $\kappa \geq 0.60$. Landis and Koch describe 0.41–0.60 as moderate agreement and 0.61–0.80 as substantial (Landis and Koch, 1977).

### 2.4.9 Confidence intervals for proportions

Nearly every number we report is a proportion, such as the share of hits that are false. A proportion measured on few items is uncertain, and we report the uncertainty with a Wilson interval (Wilson, 1927):

$$[\ \hat{p} + z^2/(2n) \pm z \cdot \sqrt{(\ \hat{p}(1 - \hat{p})/n + z^2/(4n^2)\ )}\ ]\ /\ (\ 1 + z^2/n\ )$$

Wilson intervals behave better than the normal approximation for small samples and proportions near zero or one (Brown et al., 2001). Ten-item strata remain imprecise, so their rates are descriptive.

## 2.5 Literature review

The review groups prior work by system design and evaluation focus; reported figures below retain the definitions used by each source.

### 2.5.1 Semantic caching as a system

GPTCache established an open-source response cache and reports 2–10× faster responses on a hit (Bang, 2023). GPT Semantic Cache reports up to 68.8% fewer API calls and positive-hit rates above 97% (Regmi and Pun, 2024). SCALM uses real chat data and reports 63% higher hit ratio and 77% higher token savings than GPTCache (Li et al., 2024); MeanCache reports improved F-score and precision with federated, user-side matching (Gill et al., 2025a). These studies emphasise savings and aggregate precision on their own workloads.

The systems differ in cache placement, data, and metrics; none of those aggregate results identifies which typed near-miss a static threshold admits.

### 2.5.2 Better similarity for caching

The Redis study fine-tunes compact embeddings on Quora and medical question pairs, reporting medical precision from 78% to 92% and 87% for a model trained on synthetic pairs

versus 85% for text-embedding-3-large (Gill et al., 2025b). It motivates using fixed, off-the-shelf encoders as a baseline rather than assuming one matcher is universally best.

### 2.5.3 Context-aware and multi-turn caching

ContextCache incorporates dialogue context and reports roughly 10× lower latency than a direct call (Yan et al., 2025). SmartCache models conversational turns and reports reductions in memory and time-to-first-token on CoQA and SQuAD (Yu et al., 2025). These systems address history-dependent queries; the present single-turn design isolates threshold behaviour.

### 2.5.4 Treating correctness as the quantity to control

vCache uses a policy to meet user-defined error-rate guarantees and reports improvements over static thresholds and fine-tuned embeddings (Schroeder et al., 2026). Krites adds asynchronous judging and reports increases in requests served from curated answers, using an oracle based on benchmark equivalence classes; its separate judge check agreed with labels on 99/100 borderline pairs (Singh et al., 2026).

Both make correctness central, but they control errors through policy or curation. This study instead measures the errors of fixed matchers under controlled query changes, and separately evaluates whether a stored response answers the incoming query.

### 2.5.5 Neighbouring work on cost, and on testing with perturbations

FrugalGPT uses prompt adaptation, model approximation, and cascades, reporting up to 98% lower cost at comparable performance (Chen et al., 2023). RAGCache and Prompt Cache reuse internal states rather than whole responses (Jin et al., 2024; Gim et al., 2023). These techniques reduce generation work but do not pose the same risk of returning an answer to a different question.

PAWS, contrast sets, and CheckList test models with targeted perturbations and reveal failures hidden by aggregate scores (Zhang et al., 2019; Gardner et al., 2020; Ribeiro et al., 2020). Truong et al. report weaknesses in handling negation (2023). These resources motivate typed hard negatives, but do not evaluate cache hits, stored responses, or thresholds.

*TABLE 2.3. The reviewed work along four dimensions.*

| Work | Level of reuse | What its abstract reports | Treatment of the threshold, as stated |
|---|---|---|---|
| GPTCache (Bang, 2023) | Response | Speed (2–10× on a hit) | Not the subject of the abstract |

| Work | Level of reuse | What its abstract reports | Treatment of the threshold, as stated |
|---|---|---|---|
| GPT Semantic Cache (Regmi and Pun, 2024) | Response | API calls reduced up to 68.8%; positive hit rate above 97% | Not the subject of the abstract |
| SCALM (Li et al., 2024) | Response | Hit ratio and token savings vs. GPTCache | Not the subject of the abstract |
| MeanCache (Gill et al., 2025a) | Response | F-score, precision, storage, decision speed vs. GPTCache | Learned similarity model |
| Redis report (Gill et al., 2025b) | Response | Precision gains from fine-tuned embeddings | Not the subject of the abstract |
| ContextCache (Yan et al., 2025) | Response, multi-turn | About 10× lower latency | Not the subject of the abstract |
| SmartCache (Yu et al., 2025) | Attention (key-value) state, multi-turn | Memory, time-to-first-token and answer quality | Not the subject of the abstract |
| vCache (Schroeder et al., 2026) | Response | Meets user-defined error bounds | Replaced by an error-bounded policy |
| Krites (Singh et al., 2026) | Curated response | Share of requests served from curated answers | Judge verifies borderline matches |
| FrugalGPT (Chen et al., 2023) | None (model selection) | Cost reduction up to 98% | Not applicable |
| RAGCache (Jin et al., 2024) | Internal state of retrieved documents | Time-to-first-token, throughput | Not applicable |
| Prompt Cache (Gim et al., 2023) | Internal attention state | Time-to-first-token | Not applicable |

## 2.6 Synthesis and the research gap

Prior cache studies report aggregate savings, precision, or policy guarantees, while perturbation benchmarks test targeted changes without a response cache. The reviewed work does not report static-threshold false-hit rates separately for entity, negation, number/date, and scope changes.

The resulting gaps are typed threshold errors (RQ1), judge–label disagreement on reused responses (RQ2), a cache-oriented audited corpus (RQ3), and a matched-cost comparison of matchers (RQ4). Chapter 3 describes the evaluation.

We state the gap carefully. We reviewed the abstracts and pages of the papers cited here, and we do not claim that no other work exists. The claim is limited to the material we reviewed.

# Chapter 3 Topic Description and Work Performed

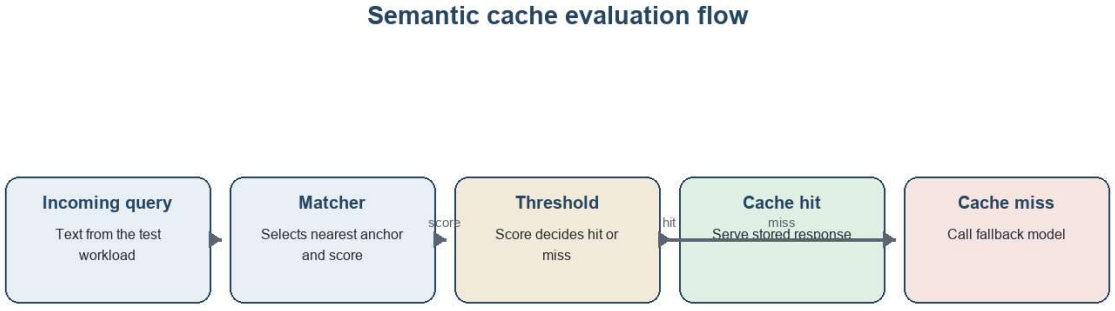
This chapter describes what we built and how we ran it. Sections 3.1 to 3.9 cover the design: the problem setting, the data, the way the evaluation corpus is made and checked, the matching methods, the judge, the metrics and the experimental protocol. Section 3.10 reports the analysis and the results.

## 3.1 Problem setting and overview

A static semantic cache stores anchor questions with responses. For each incoming query, a matcher selects the nearest anchor and score; a hit above τ reuses its response, while a miss calls the fallback model. Misses do not alter the cache, so each query is independent of arrival order.

The evaluation corpus labels each query by construction as a paraphrase, typed hard negative, or unrelated query. Four matchers score the same corpus. Expensive outputs are stored once and threshold sweeps replay those scores; Fig. 3.1 shows the pipeline.

*Fig. 3.1. End-to-end pipeline.*




## 3.2 Dataset

### 3.2.1 Source of the questions

Anchors are drawn from NQ-Open, the open-domain Natural Questions dataset (Lee et al., 2019), which uses anonymised Google Search queries (Kwiatkowski et al., 2019).

### 3.2.2 What the evaluation corpus contains

From the anchors we make four kinds of query. Table 3.1 lists them with the intended correct behaviour of a cache.

*TABLE 3.1. Kinds of evaluation query.*

| Kind | Code | Relation to its anchor | Correct behaviour of the cache |
|---|---|---|---|
| Paraphrase, light rewording | P1 | Same question, small change of wording | Hit on its own anchor |
| Paraphrase, moderate rewording | P2 | Same question, larger change | Hit on its own anchor |
| Paraphrase, strong rewording | P3 | Same question, much of the wording replaced | Hit on its own anchor |
| Paraphrase, heaviest rewording | P4 | Same question, heaviest rewording | Hit on its own anchor |
| Entity hard negative | E | Same shape, a different named entity | Miss |
| Negation hard negative | N | Same shape, opposite truth condition | Miss |
| Numeric or temporal hard negative | T | Same shape, a different quantity, date or range | Miss |
| Scope hard negative | S | Same shape, a narrower or broader subject | Miss |
| Unrelated | U | A different question altogether | Miss |

The four paraphrase levels P1 to P4 are intended levels of difficulty, from light word substitution to heavy rewording. We do not assume the levels are ordered as intended. For every paraphrase we measure how far it actually is from its anchor, and we check the ordering afterwards (Section 3.4.2).

## 3.3 Preprocessing

### 3.3.1 Mechanical filter

Raw questions are not all usable. A mechanical filter removes records that have an empty question, control characters, no alphabetic character, a web address or markup, a number of words outside a fixed range, or one word that makes up most of the question, and records for which the dataset provides no usable answer. The filter is deliberately simple and rule-based, so anyone can predict what it will remove.

### 3.3.2 Tokenisation

Several later steps compare questions as sets of words, so they need one definition of a word. We lowercase the text and take each run of letters and digits as a token. Punctuation and underscores separate tokens. For example, "Who wrote Hamlet?" gives the three tokens who, wrote and hamlet.

### 3.3.3 Near-duplicate removal

Two questions in the data can be nearly the same. If one became an anchor and the other became an "unrelated" query, the corpus would contain a labelling error: the unrelated query would deserve a hit. Deduplication is also known to matter for a wider reason, since duplicated text inflates results when the same content sits on both sides of a comparison (Lee et al., 2022). We therefore remove near-duplicates using the Jaccard similarity of token sets (Section 2.4.5), with a fixed threshold set in the configuration, and keep the first occurrence of each group. Comparing every pair of questions would be slow, so we use a prefix filter. It indexes only the rarest tokens of each question, so most pairs are never compared, while every pair that could reach the threshold is still found and checked exactly (Chaudhuri et al., 2006).

Worked example. Take the question "who sang the song hello" (five tokens) and "who sang the song hello in 2015" (seven tokens). They share five tokens and their union has seven, so their Jaccard similarity is 5/7 = 0.714. Under any threshold at or below that value, the second question would be removed as a near-duplicate of the first, because the first came earlier.

### 3.3.4 Seeded sampling

After filtering and deduplication, we draw from the remaining questions with a fixed random seed (42), so the same draw can be repeated exactly. The draw gives three groups:

- 100 anchors, which form the cache.
- 50 development anchors, which are used only to debug prompts and the pipeline. They never enter the cache or any reported number.
- A held-out pool of 1,000 questions, from which the unrelated queries are chosen.

The 100 unrelated queries are taken from the held-out pool. Each must have a token-level Jaccard similarity of at most 0.2 with every anchor, so that an unrelated query cannot be a disguised paraphrase.

## 3.4 Corpus construction

### 3.4.1 Generating paraphrases and hard negatives

A local language model, Gemma 3 with 4 billion parameters, writes the variants (Gemma Team, 2025; Table 2.2). For each anchor it writes four paraphrases, one for each of P1 to P4. It also writes hard negatives of the four types E, N, T and S, up to 50 per type. The four types are chosen because each is a common way in which two questions can look alike and still need different answers.

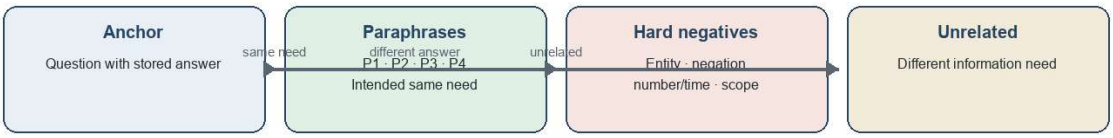
*TABLE 3.2. Hard-negative types, with illustrative pairs written for this report (not taken from the corpus).*

| Type | Definition | Anchor | Hard negative |
|---|---|---|---|
| E, entity substitution | Same question shape, a different named entity | who directed the film Inception | who directed the film Interstellar |
| N, negation or polarity flip | Same shape, opposite truth condition | which planets have rings | which planets do not have rings |
| T, numeric or temporal change | Same shape, a different quantity, date or range | who won the FIFA World Cup in 2018 | who won the FIFA World Cup in 2014 |
| S, scope change | Same shape, a narrower or broader subject | what is the population of Jaipur | what is the population of Rajasthan |

Not every perturbation fits every anchor. A question with no number cannot be given a numeric change in a natural way. We allow the generator to answer that the perturbation does not apply, and then we skip that anchor for that type. We never force a perturbation, because a forced hard negative is likely to be a bad one. This is also why we say "up to" 50 per type.

Every generated query passes a validity check. It is rejected if it is not valid output, if it equals the anchor after normalisation or equals any paraphrase of the same anchor, or if its length differs from the anchor's by more than a factor of two. A rejected query is regenerated, up to three times. Each query is tagged with its kind, so every result can later be broken down by kind.

*Fig. 3.2. The structure of the corpus around one anchor.*




### 3.4.2 Two measured covariates

Whether a paraphrase is "light" or "heavy" is the generator's intention, not a fact. For every query we therefore record two measures against its anchor: the token-level Jaccard similarity and the normalised edit distance (Sections 2.4.5 and 2.4.6). A high Jaccard similarity and a low edit distance both mean "close in wording". These measures let us check whether the intended levels really are ordered by distance, and they let us study false misses against measured distance and not only against the label. We treat them as descriptions of surface wording and nothing more. They are not measures of meaning.

### 3.4.3 Stored responses and the size of the corpus

For each anchor, the fallback model, Llama 3.2 with 3 billion parameters (Meta AI, 2024), writes one response at temperature 0, and that response is stored in the cache. Temperature 0 makes the output as repeatable as the software allows. The corpus then holds 400 paraphrases (100 for each of P1 to P4), at most 200 hard negatives (at most 50 for each of E, N, T and S) and 100 unrelated queries, so at most 700 evaluation queries. The unrelated queries are a floor check and not a discriminating test: any hit on one is a false hit.

## 3.5 Label verification

We assess labels on a stratified audit of 90 queries: ten from each of P1–P4, E, N, T, S, and unrelated. Several independent LLM raters—ChatGPT GPT-6 Luna, Sonnet 5.5, and Claude Sonnet 4.6 (Thinking)—saw the query, matched anchor, and cached response without sight of the intended label and supplied yes/no ratings. We compare their saved majority acceptability label with Phi-4-mini. Thirty-seven paraphrase records that initially failed processing were

recovered before the final 700-query corpus was assembled; the audit sample was drawn before recovery.

## 3.6 Matching methods: the process model

### 3.6.1 What a matcher does

Every matching method does the same job. Given a query q, it returns a candidate anchor a*(q) and a score s(q). The cache has a hit at threshold τ when s(q) is at least τ, and a miss otherwise:

$$\text{hit}_\tau(q) = 1 \text{ if } s(q) \geq \tau, \text{ otherwise } 0 \qquad (3.1)$$

On a hit, the stored response of a(q) is served. On a miss, the fallback model is called. The four methods differ only in how they compute a(q) and s(q).

*TABLE 3.3. The four matching configurations.*

| Configuration | Candidate and score |
|---|---|
| Bi-encoder with all-MiniLM-L6-v2 | The largest cosine similarity over all anchors; the vectors have 384 dimensions |
| Bi-encoder with bge-base-en-v1.5 | The largest cosine similarity over all anchors; the vectors have 768 dimensions |
| Bi-encoder with cross-encoder reranking | The five closest anchors under all-MiniLM-L6-v2 are re-scored by a cross-encoder; s(q) is the highest score and a*(q) its anchor |
| Lexical | The difflib sequence ratio between lowercased texts; the anchor with the highest ratio is the candidate |

### 3.6.2 Bi-encoder matching

A bi-encoder turns each question into a vector independently, so all anchor vectors can be computed once and stored (Reimers and Gurevych, 2019). The steps are:

19. Encode every anchor and normalise each vector to length 1.
20. Place the vectors in an exact (flat) FAISS index (Johnson et al., 2021). For vectors of length 1, the inner product equals the cosine similarity.
21. For each query, encode it, normalise it and ask the index for the single nearest anchor. Its inner product is s(q) and its anchor is a*(q).

We use an exact index on purpose. It returns the true nearest anchor every time, so no result reflects the approximation error that approximate indexes can introduce (Malkov and Yashunin, 2020). With only 100 anchors, exact search costs almost nothing. The two embedding models are all-MiniLM-L6-v2, built on the MiniLM architecture (Wang et al.,

2020), and bge-base-en-v1.5, from the BGE family (Xiao et al., 2023); the BGE authors report state-of-the-art results for their English models on the MTEB benchmark at the time of release (Xiao et al., 2023; Muennighoff et al., 2023).

#### 3.6.3 Reranking with a cross-encoder

The third configuration follows the retrieve-then-rerank pattern, in which a cheap retriever proposes candidates and a more expensive model, which reads the query and a candidate together, re-scores them (Nogueira and Cho, 2019). We use MiniLM to propose the five nearest anchors and a cross-encoder trained on duplicate-question pairs (Cross-Encoder, 2026) to re-score each of the five. The score s(q) is the highest cross-encoder score, a number between 0 and 1, and a*(q) is the anchor that earned it. One consequence follows from the design: if a paraphrase's own anchor is not among the five MiniLM proposals, the cross-encoder cannot find it.

#### 3.6.4 Lexical matching

The fourth configuration has no learned component. It computes the difflib sequence ratio between the lowercased query and each anchor, and takes the anchor with the highest ratio. It shows what surface similarity alone can achieve, and it is a useful contrast because it responds to wording and not to meaning.

#### 3.6.5 A note on comparing scores

The three kinds of score do not live on a common scale. A cosine similarity, a cross-encoder probability and a sequence ratio can all be written between 0 and 1, but a value of 0.75 means something different in each. For this reason, any comparison across matchers must be read at a matched cost saving and not only at a matched threshold, and we plot the correctness-cost frontier for that purpose.

### 3.7 The judge

#### 3.7.1 Task and model

For every pair of a query and the anchor it matched, a language-model judge receives the incoming query, the matched anchor and the stored response, and returns one of two verdicts: acceptable or unacceptable for the incoming query. The judge is Phi-4-mini with 3.8 billion parameters, run locally at temperature 0 (Microsoft, 2025). We judge the reused response and not only the pair, because a matched pair can carry the right label while the particular stored response still does not answer the new query.

### 3.7.2 Why the judge comes from a different family

A judge that shares a family with the model that wrote the text it grades tends to favour that text (Panickssery et al., 2024; Liu et al., 2023). The corpus generator is Gemma 3 (Gemma Team, 2025), the fallback model is Llama 3.2 (Meta AI, 2024) and the judge is Phi-4-mini (Microsoft, 2025), so no judge shares a family with the writer of anything it grades.

### 3.7.3 Checking the judge

Strong language-model judges have shown substantial agreement with preference labels, but this does not establish reliability for a small local judge (Zheng et al., 2023). We therefore fixed two validation gates before analysis: anchor self-acceptance of at least 90% and Cohen's κ of at least 0.60 against the 90-item audit consensus.

22. The anchor check runs the judge on each of the 100 anchors with its own stored response. It also reflects response quality and is a pipeline gate, not a stand-alone accuracy estimate.
23. For the audit check, compare the judge with the raters' majority label for response acceptability on 90 items using Cohen's κ (Cohen, 1960; Landis and Koch, 1977).

If both gates pass, JFH is primary; otherwise construction-label FHR and hard-negative hit rate are primary, with JFH and measured judge errors secondary. The unchanged prompt and verdicts are stored in Appendix A and the project records.

## 3.8 Metrics

### 3.8.1 Judged false-hit rate

Among the hits at threshold τ, the judged false-hit rate is the share whose reused response the judge finds unacceptable:

$$JFH(\tau) = |\{q : hit_\tau(q) = 1 \text{ and the judge says unacceptable}\}| \,/\, |\{q : hit_\tau(q) = 1\}| \qquad (3.2)$$

### 3.8.2 Label-based rates

Each hit is also scored against the construction label and the identity of the matched anchor, with no judge involved. A hit is a true hit (TH) if the query is a paraphrase and its nearest anchor is its own source anchor. Every other hit is a label false hit (FH). That includes a hard negative, an unrelated query, and a paraphrase that lands on the wrong anchor. The label false-hit rate among hits, and the false-hit incidence over all queries, are:

$$FHR(\tau) = FH(\tau) \,/\, (TH(\tau) + FH(\tau)) \qquad (3.3)$$

$$QFHI(\tau) = FH(\tau) \,/\, |Q| \qquad (3.4)$$

QFHI is the share of all queries that are label-false hits. Hard-negative hit rate is the share of queries of a given hard-negative type served from cache; it is not conditional on hits.

### 3.8.3 False misses and cost

Each paraphrase query has its anchor in the cache, so a miss on a paraphrase is a false miss. With Q_para the set of paraphrase queries:

$$FMR(\tau) = |\{q \text{ in } Q_para : hit_\tau(q) = 0\}| / |Q_para| \quad (3.5)$$

Each miss triggers exactly one call to the fallback model, so the saving relative to a system with no cache is the share of queries that never reach the model:

$$CS(\tau) = 1 - |\{q : hit_\tau(q) = 0\}| / |Q| \quad (3.6)$$

Cost is therefore counted as model calls avoided. We also report the mean stored fallback latency of the queries that were served from the cache. This is the work avoided before lookup time is counted, and so it is not a net serving-time saving. We do not turn these quantities into prices.

### 3.8.4 Judge-label disagreement

Judge–label disagreement includes accepting a label-false hit or rejecting a true hit. The measures answer different questions: construction labels test retrieval of the source anchor, while the judge tests response acceptability; a label-false scope or numeric change may still be answered by the stored response.

### 3.8.5 Uncertainty

Every rate above is a proportion k out of n, and we report it with a Wilson 95% interval (Section 2.4.9) (Wilson, 1927; Brown et al., 2001). The interval treats queries as independent. The paraphrases and hard negatives of one anchor are not independent, so the reported intervals are somewhat narrower than the true uncertainty.

### 3.8.6 Comparison at matched savings

Because score scales differ, compare configurations at equal hit share: rank the 700 stored scores within each matcher and retain the top 280 (40%). The resulting cut-off is matcher-specific. This descriptive in-sample point is chosen without reference to error outcomes and is not a deployment threshold.

## 3.9 Experimental setup and protocol

### 3.9.1 Hardware and software

All experiments ran on one consumer machine with an AMD Ryzen 5 processor and integrated graphics and no discrete graphics card, so every model ran on the processor. The three generative models were served locally through Ollama and the embedding models and the cross-encoder were loaded through the sentence-transformers library (Section 2.2).

*TABLE 3.4. Settings used in the experiments.*

| Component | Setting |
|---|---|
| Anchor source | NQ-Open, pinned revision |
| Corpus generator | Gemma 3, 4 billion parameters |
| Fallback model and stored responses | Llama 3.2, 3 billion parameters, temperature 0 |
| Judge | Phi-4-mini, 3.8 billion parameters, temperature 0 |
| Bi-encoders | all-MiniLM-L6-v2 and bge-base-en-v1.5 |
| Cross-encoder | cross-encoder/quora-distilroberta-base |
| Vector index | FAISS flat inner-product index over normalised vectors |
| Lexical matcher | difflib sequence ratio |
| Threshold grid | 0.50 to 0.95 in steps of 0.05 (ten values) |
| Seed | 42 for all sampling |
| Evaluation set | 400 paraphrases, at most 200 hard negatives, 100 unrelated queries |

### 3.9.2 The three stages

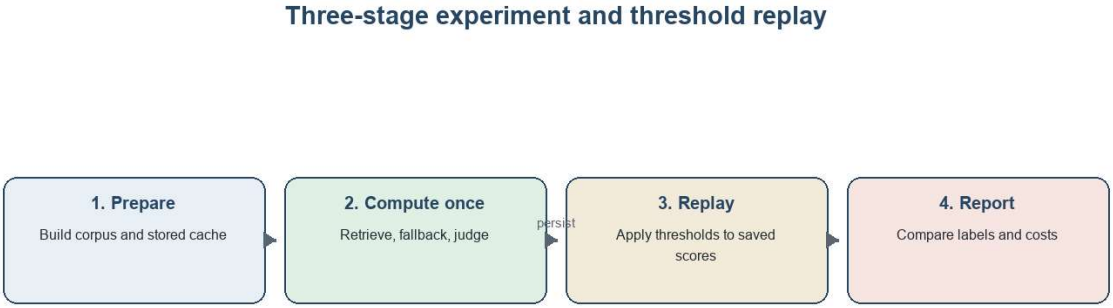
Stage 1: build the corpus. The questions are filtered, deduplicated and sampled, the variants are generated and checked, and the cache is filled with the anchors and their stored responses (Sections 3.3 and 3.4).

Stage 2: compute once. For every query and every configuration, the candidate anchor and its score are computed once. The fallback model is called once for every query, and the response, its latency and its length are stored. The judge is called once for every pair of a configuration and a query, whatever the threshold, and the verdict is stored. A pair is never judged twice.

Stage 3: replay. Thresholds are applied afterwards to the stored scores. At each $\tau$ in the grid, a query is a hit when its score reaches $\tau$. A miss is answered with the stored fallback response

and latency. The matched-savings comparison of Section 3.8.6 is replayed in the same way. The sweep needs no model calls, and running it again gives exactly the same numbers. The label audit and the judge gates (Sections 3.5 and 3.7.3) are applied to the stored records afterwards.

*Fig. 3.3. The replay design.*




### 3.9.3 Experiments

The main experiment applies the threshold grid to all four configurations on the same queries, the same cache and the same judge, so that any difference between curves comes from the matching method. Every rate is computed overall and by query kind: the four paraphrase levels, the four hard-negative types and the unrelated set. The gap between the judged and the label-based measures is reported by kind. The matcher comparison is made at exactly 40% model-call savings (Section 3.8.6), and the within-matcher threshold curves are shown separately. Both are summaries of the evaluated corpus and not estimates of what a deployed cache would reach.

### 3.9.4 Determinism and reproducibility

The corpus is generated once with a fixed seed and every model call uses temperature 0, so the experiment is one deterministic run, and no variation across seeds is estimated. Dataset sampling, filtering, deduplication, the sweep and every metric are deterministic given the configuration. The generated paraphrases, hard negatives and responses depend on the exact model builds, so they are kept as stored files and are not expected to regenerate identically on another machine. The released material consists of the evaluation corpus with its labels and covariates, the filtered anchor list, the stored responses and latencies, the judge verdicts, the

audit sheet and a record of the model builds. Because the sweep replays stored results, every table and figure can be regenerated without running a model.

## 3.10 Analysis and results

Results use the saved 700-query corpus and stored retrieval, fallback, and judge records; threshold analysis makes no new model calls. The judge fails both prespecified gates, so construction-label measures are primary and JFH is secondary. Proportions use 95% Wilson intervals (Section 3.8.5).

### 3.10.1 The evaluation set

The evaluation set holds 400 paraphrases (100 at each level), 200 hard negatives (50 of each type) and 100 unrelated queries. The 100 anchors are the same for every configuration. Table 3.5 gives the measured covariates of Section 3.4.2. For paraphrases and hard negatives they are measured against the source anchor, and for unrelated queries against the anchor with the largest token overlap.

*TABLE 3.5. Composition of the evaluation set and mean covariates.*

| Kind | n | Mean token Jaccard | Mean normalised edit distance |
|---|---|---|---|
| P1 | 100 | 0.4175 | 0.4472 |
| P2 | 100 | 0.3517 | 0.5549 |
| P3 | 100 | 0.3432 | 0.5600 |
| P4 | 100 | 0.2814 | 0.6211 |
| Entity (E) | 50 | 0.3102 | 0.5714 |
| Negation (N) | 50 | 0.7371 | 0.1852 |
| Numeric or temporal (T) | 50 | 0.3079 | 0.5870 |
| Scope (S) | 50 | 0.2791 | 0.6181 |
| Unrelated | 100 | 0.1543 | 0.6957 |

P1 is closest to its anchor and P4 farthest, but P2 and P3 are nearly indistinguishable (mean Jaccard 0.352 vs 0.343; edit distance 0.555 vs 0.560). P1's mean Jaccard is 0.42, so its "light" label is not a lexical band. Negation retains substantial wording (Jaccard 0.74; edit distance 0.19), while unrelated queries average 0.15 Jaccard. We therefore interpret results by construction type and measured distance separately.

### 3.10.2 Behaviour across the threshold grid

Fig. 3.4 shows the secondary judged false-hit rate over each configuration's own threshold grid; Fig. 3.5 shows the primary construction-label correctness–savings frontier at matched savings.

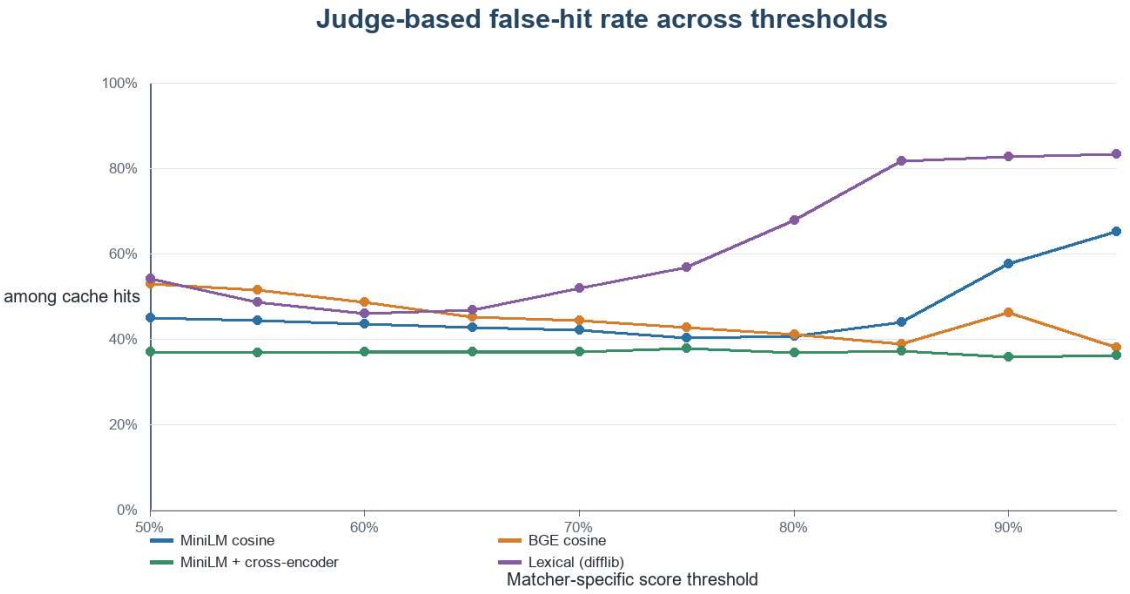



*Fig. 3.4. Judged false-hit rate among hits over each configuration's threshold grid (secondary diagnostic).** The denominator is the number of hits for that configuration and threshold.*

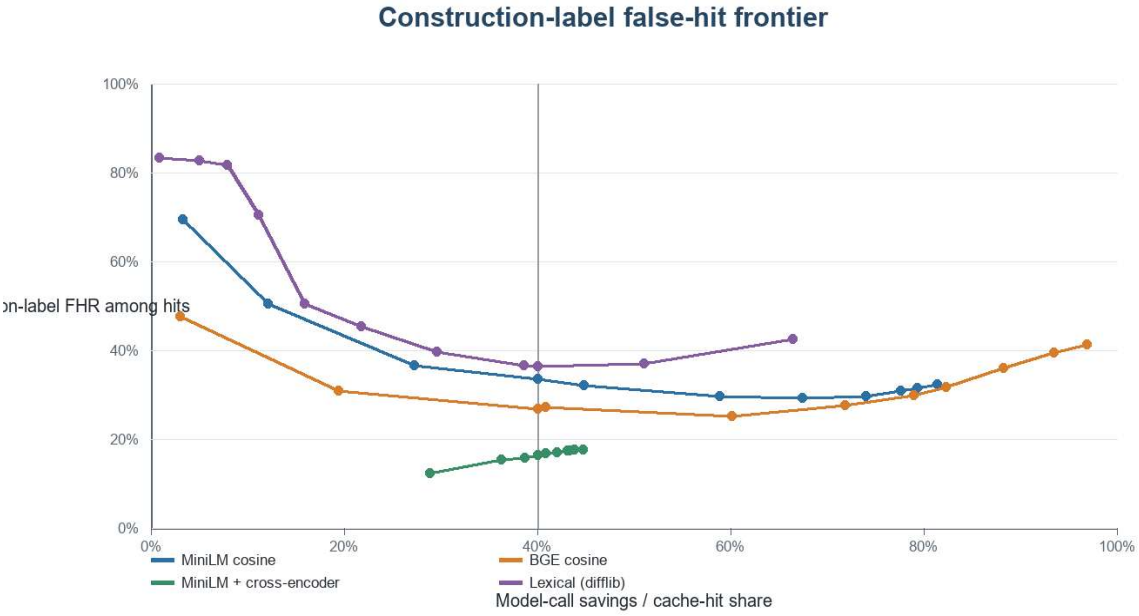



*Fig. 3.5. Construction-label FHR against model-call savings at the tested thresholds; markers show an exact matched 40% hit-share point.*

For the cross-encoder, secondary JFH stays between 35.8% and 37.8% over $\tau = 0.50–0.90$. Because the judge fails validation, this apparent flatness is a property of the judged measure, not evidence of flat cache error.

### 3.10.3 Comparison at matched model-call savings

To compare matchers fairly, we rank the 700 stored scores within each configuration and keep exactly the top 280 queries as hits, which gives a 40% hit share for every matcher (Section 3.8.7). The cut-off that results differs from matcher to matcher. Table 3.6 gives the results.

*TABLE 3.6. The four configurations at exactly 40% model-call savings.*

| Matcher | Score cut-off | Hit calls / savings | Label FHR among hits (95% CI) | Query false-hit incidence | Paraphrase false-miss rate | JFH (95% CI; secondary) |
|---|---|---|---|---|---|---|
| MiniLM cosine | 0.8163 | 280/700 (40.0%) | 94/280 (33.6%; 28.3–39.3%) | 94/700 (13.4%) | 214/400 (53.5%) | 116/280 (41.4%; 35.8–47.3%) |
| BGE cosine | 0.8526 | 280/700 (40.0%) | 75/280 (26.8%; 21.9–32.3%) | 75/700 (10.7%) | 195/400 (48.8%) | 108/280 (38.6%; 33.1–44.4%) |
| MiniLM + cross-encoder | 0.8009 | 280/700 (40.0%) | 46/280 (16.4%; 12.5–21.2%) | 46/700 (6.6%) | 166/400 (41.5%) | 103/280 (36.8%; 31.4–42.6%) |
| Lexical (difflib) | 0.5941 | 280/700 (40.0%) | 102/280 (36.4%; 31.0–42.2%) | 102/700 (14.6%) | 220/400 (55.0%) | 130/280 (46.4%; 40.7–52.3%) |

At 40% hit share, cross-encoder reranking has the lowest construction-label FHR (16.4%), followed by BGE (26.8%), MiniLM (33.6%), and lexical matching (36.4%). Its paraphrase FMR is also substantial at 41.5%. Wilson intervals overlap for some pairs; these descriptive intervals are not paired significance tests and may be too narrow because queries share anchors. The ranking is workload-specific and in-sample.

### 3.10.4 What each kind of variation does to a threshold

At $\tau = 0.75$, within each matcher's own scale, the share of the 50 negation queries that are served from the cache is 49 (98%) for MiniLM and for BGE, 46 (92%) for the lexical matcher and 12 (24%) for the cross-encoder. These are within-matcher results on the shared grid, not comparisons at a common operating point. At the matched 40% point, Table 3.7

gives the hard-negative hit rate for each type, together with the judged false-hit rate within the hits of that type as a secondary view.

*TABLE 3.7. Hard-negative hit rate and secondary JFH by kind of query at exactly 40% model-call savings.*

| Kind | MiniLM cosine | BGE cosine | Cross-encoder | Lexical (difflib) |
|---|---|---|---|---|
| P1 | JFH 25/63 (39.7%) | JFH 24/67 (35.8%) | JFH 29/72 (40.3%) | JFH 27/68 (39.7%) |
| P2 | JFH 18/58 (31.0%) | JFH 21/63 (33.3%) | JFH 25/66 (37.9%) | JFH 21/47 (44.7%) |
| P3 | JFH 7/27 (25.9%) | JFH 10/44 (22.7%) | JFH 18/57 (31.6%) | JFH 12/44 (27.3%) |
| P4 | JFH 13/38 (34.2%) | JFH 13/31 (41.9%) | JFH 17/39 (43.6%) | JFH 11/21 (52.4%) |
| E | 17/50 hit (34%); JFH 5/17 (29%) | 11/50 hit (22%); JFH 1/11 (9%) | 12/50 hit (24%); JFH 3/12 (25%) | 18/50 hit (36%); JFH 7/18 (39%) |
| N | 45/50 hit (90%); JFH 38/45 (84%) | 40/50 hit (80%); JFH 34/40 (85%) | 12/50 hit (24%); JFH 7/12 (58%) | 48/50 hit (96%); JFH 41/48 (85%) |
| T | 20/50 hit (40%); JFH 5/20 (25%) | 15/50 hit (30%); JFH 3/15 (20%) | 14/50 hit (28%); JFH 2/14 (14%) | 17/50 hit (34%); JFH 3/17 (18%) |
| S | 12/50 hit (24%); JFH 5/12 (42%) | 9/50 hit (18%); JFH 2/9 (22%) | 8/50 hit (16%); JFH 2/8 (25%) | 14/50 hit (28%); JFH 5/14 (36%) |
| Unrelated | 0/100 hit; JFH not defined | 0/100 hit; JFH not defined | 0/100 hit; JFH not defined | 3/100 hit; JFH 3/3 (100%) |

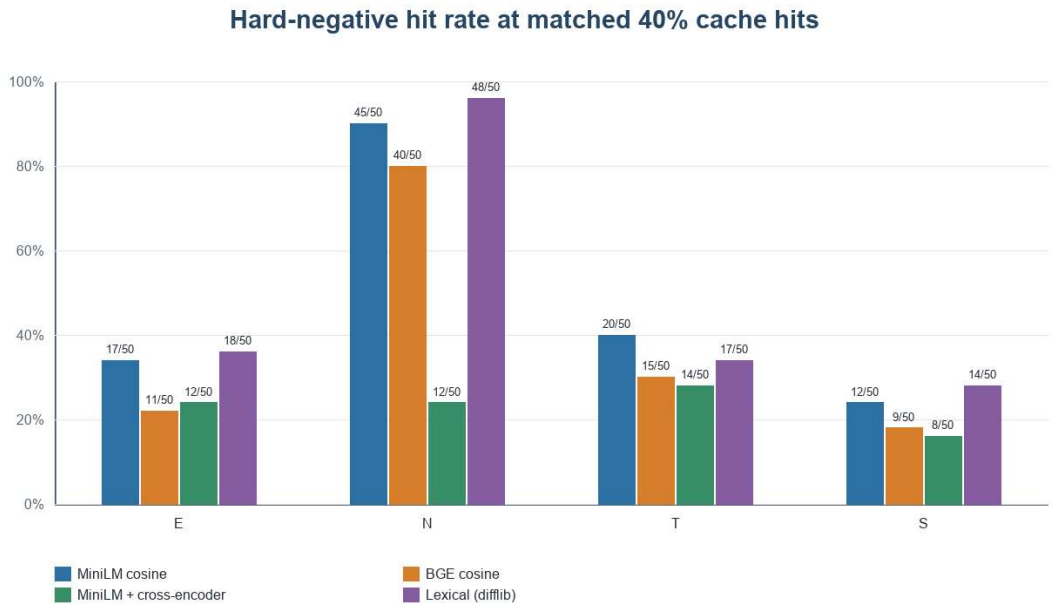



*Fig. 3.6. Share of the queries of each hard-negative type that are served from the cache, at exactly 40% model-call savings.** Bars are 95% Wilson intervals. Every hard-negative hit is a label false hit.*

Negation passes most often: at $\tau = 0.75$, the hard-negative hit rates are 98% for each bi-encoder, 92% for lexical matching, and 24% for the cross-encoder. At matched 40% savings, 45 MiniLM, 40 BGE, 48 lexical, and 12 cross-encoder negation queries are served. Negation's high mean Jaccard (0.74) explains why surface similarity admits it. The cross-

encoder, trained on duplicate-question pairs rather than response acceptability, suppresses negation but still misses 41.5% of paraphrases at the matched point. It also has the lowest hit rates for E, T, and S. This pattern is consistent with joint pair scoring being sensitive to "not," but that explanation was not experimentally isolated. Some label-false scope or numeric hits may still be acceptable answers.

### 3.10.5 False misses, and wording distance

A false miss is a paraphrase that the cache fails to serve. Table 3.8 gives the false-miss rate at the matched 40% point, by paraphrase level and by the two measured distances.

*TABLE 3.8. Paraphrase false-miss rate by level and by measured wording distance, at exactly 40% model-call savings.*

| Measure | Bin | n | MiniLM cosine | BGE cosine | MiniLM + cross-encoder | Lexical (difflib) |
|---|---|---|---|---|---|---|
| Paraphrase level | P1 | 100 | 37/100 (37%) | 33/100 (33%) | 28/100 (28%) | 32/100 (32%) |
| Paraphrase level | P2 | 100 | 42/100 (42%) | 37/100 (37%) | 34/100 (34%) | 53/100 (53%) |
| Paraphrase level | P3 | 100 | 73/100 (73%) | 56/100 (56%) | 43/100 (43%) | 56/100 (56%) |
| Paraphrase level | P4 | 100 | 62/100 (62%) | 69/100 (69%) | 61/100 (61%) | 79/100 (79%) |
| Token Jaccard | 0.0–0.2 | 71 | 59/71 (83.1%) | 57/71 (80.3%) | 51/71 (71.8%) | 70/71 (98.6%) |
| Token Jaccard | 0.2–0.4 | 181 | 113/181 (62.4%) | 101/181 (55.8%) | 89/181 (49.2%) | 119/181 (65.7%) |
| Token Jaccard | 0.4–0.6 | 118 | 38/118 (32.2%) | 37/118 (31.4%) | 22/118 (18.6%) | 27/118 (22.9%) |
| Token Jaccard | 0.6–0.8 | 26 | 3/26 (11.5%) | 0/26 (0.0%) | 3/26 (11.5%) | 4/26 (15.4%) |
| Token Jaccard | 0.8–1.0 | 4 | 1/4 (25.0%) | 0/4 (0.0%) | 1/4 (25.0%) | 0/4 (0.0%) |
| Edit distance | 0.0–0.2 | 10 | 1/10 (10.0%) | 0/10 (0.0%) | 2/10 (20.0%) | 0/10 (0.0%) |
| Edit distance | 0.2–0.4 | 62 | 10/62 (16.1%) | 7/62 (11.3%) | 9/62 (14.5%) | 0/62 (0.0%) |
| Edit distance | 0.4–0.6 | 163 | 83/163 (50.9%) | 75/163 (46.0%) | 51/163 (31.3%) | 58/163 (35.6%) |
| Edit distance | 0.6–0.8 | 156 | 113/156 (72.4%) | 106/156 (67.9%) | 98/156 (62.8%) | 153/156 (98.1%) |
| Edit distance | 0.8–1.0 | 9 | 7/9 (77.8%) | 7/9 (77.8%) | 6/9 (66.7%) | 9/9 (100.0%) |

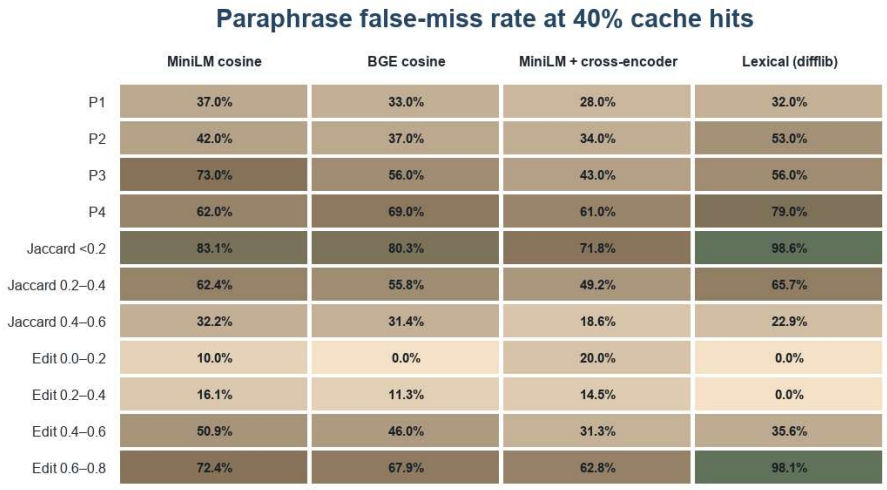



***Fig. 3.7. Paraphrase false-miss rate by paraphrase level, token Jaccard bin and edit-distance bin, at exactly 40% model-call savings.** Bars are 95% Wilson intervals.*

False misses generally rise with paraphrase difficulty, although MiniLM peaks at P3 (73%) rather than P4 (62%), consistent with the weak lexical ordering of the generated levels. Measured distance shows a clearer trend: below Jaccard 0.2, 72–99% are missed, while above 0.6 no matcher exceeds 25%. The lexical matcher misses 98.6% of the lowest-overlap group. Extreme bins are small and have wide intervals; these associations are descriptive, not causal.

### 3.10.6 Validation of the judge

Section 3.7.3 set two gates before the judge could be primary. The judge fails both.

- Anchor self-pairs. Phi-4-mini accepts only 70 of the 100 anchors with their own stored responses (70%), against the required 90%. This rate reflects both the quality of the stored responses and the judge's decision, so it is a gate on the pipeline and not a stand-alone estimate of the judge's accuracy.
- Agreement with the audit consensus. On the 90 audit items, the judge agrees with the majority label of the independent raters on 71.1% of items, and Cohen's κ is 0.416, below the required 0.60. On the scale of Landis and Koch this is at the low end of the moderate band (Landis and Koch, 1977).

Table 3.9 shows where the judge and the audit consensus disagree.

*TABLE 3.9. Phi-4-mini disagreement with the audit consensus, by stratum.*

| Stratum | Items | Disagreements | Rate |
|---|---|---|---|
| P1 | 10 | 4 | 40% |
| P2 | 10 | 4 | 40% |
| P3 | 10 | 4 | 40% |
| P4 | 10 | 1 | 10% |
| E | 10 | 2 | 20% |
| N | 10 | 3 | 30% |
| T | 10 | 5 | 50% |
| S | 10 | 3 | 30% |
| Unrelated | 10 | 0 | 0% |

With ten items per stratum, disagreement rates are coarse: the judge agrees most on unrelated queries and least on number/date changes.

The same judge, applied to the hits at the matched 40% point, disagrees with the construction label on a large share of them (Table 3.10).

*TABLE 3.10. Judge-label disagreement among hits at exactly 40% model-call savings.*

| Matcher | Disagreements / hits | Rate (95% CI) |
|---|---|---|
| MiniLM cosine | 104/280 | 37.1% (31.7–42.9%) |
| BGE cosine | 103/280 | 36.8% (31.4–42.6%) |
| MiniLM + cross-encoder | 121/280 | 43.2% (37.5–49.1%) |
| Lexical (difflib) | 110/280 | 39.3% (33.7–45.1%) |

A disagreement is either a judge that accepts the response to a label false hit, or a judge that rejects the response to a true hit. With the hit, label-false-hit, judged-unacceptable and disagreement counts known for a matcher, the two kinds can be separated by arithmetic. Table 3.11 shows the result.

*TABLE 3.11. Direction of the disagreements at exactly 40% model-call savings, derived from the counts in Tables 3.6 and 3.10.*

| Matcher | True hits rejected by the judge | Label false hits accepted by the judge |
|---|---|---|
| MiniLM cosine | 63 of 186 (33.9%) | 41 of 94 (43.6%) |
| BGE cosine | 68 of 205 (33.2%) | 35 of 75 (46.7%) |
| MiniLM + cross-encoder | 89 of 234 (38.0%) | 32 of 46 (69.6%) |
| Lexical (difflib) | 69 of 178 (38.8%) | 41 of 102 (40.2%) |

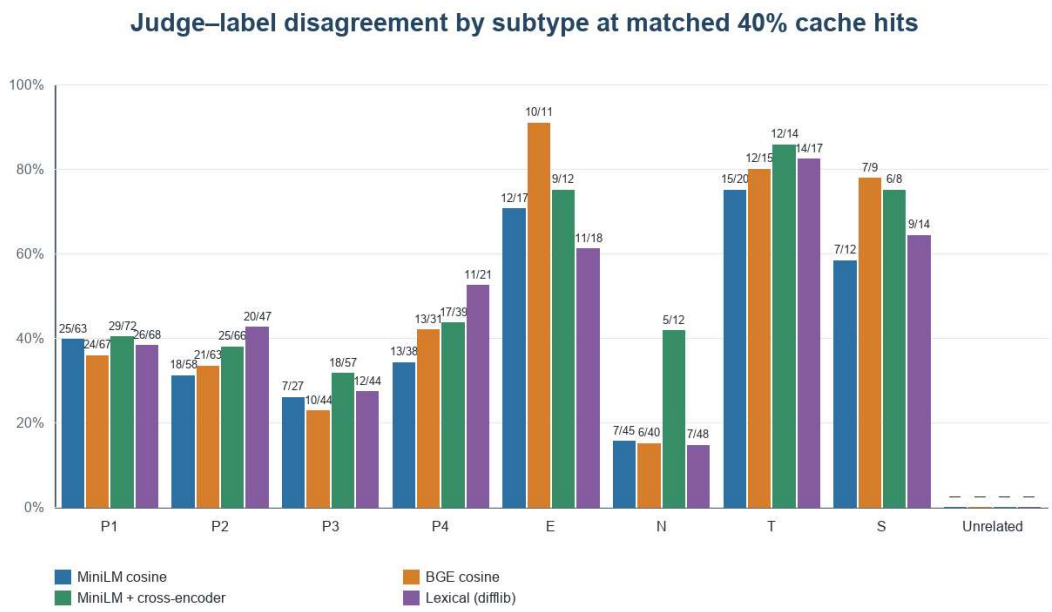



*Fig. 3.8. Judge–label disagreement by query subtype at the exact matched 40% cache-hit share.*

The judge rejects roughly one third to two fifths of true paraphrase hits and accepts about 40–70% of label-false hits (Table 3.11). This answers RQ2: on this workload the judge is not reliable enough for primary response-correctness measurement. Its errors also explain the apparent JFH floor; construction-label outcomes therefore remain primary, with judge false-reject and false-accept rates reported as diagnostics.

### 3.10.7 Comparison with published results

Published caching results use different datasets, policies and definitions of a correct hit, so their numbers cannot be set next to ours (Section 2.5). Table 3.12 lists the points that can be compared, with a caution on each.

*TABLE 3.12. Published findings on judge agreement, error control and negation, and what this work shows.*

| Prior work | Reported result or method | Relation to this study | Caution |
|---|---|---|---|
| MT-Bench (Zheng et al., 2023) | Strong judges, including GPT-4, exceed 80% pairwise preference agreement | Phi-4-mini has 71.1% raw agreement and κ = 0.416 against the audit consensus | Tasks, labels, models and samples differ |
| Krites (Singh et al., 2026) | A separate LLM judge agreed with benchmark labels on 99 of 100 borderline pairs | This study finds substantial false rejection and false acceptance by the judge on cached-response pairs | Krites uses a different benchmark and judge protocol |
| vCache (Schroeder et al., 2026) | A verified policy targets a user-defined error bound | The cross-encoder's judged rate stays between 35.8% and 37.8% across | Our judge is not validated; unlike vCache, the judged rate is not a |

| Prior work | Reported result or method | Relation to this study | Caution |
|---|---|---|---|
| | | $\tau = 0.50$–$0.90$, but the judge failed validation, so this is not evidence of a flat cache error | verified error bound |
| Truong et al. (Truong et al., 2023) | Reports language-model difficulties with negation | At $\tau = 0.75$ the bi-encoders hit 49 of 50 negation queries and the cross-encoder 12 of 50 | Our result is cache retrieval on generated queries and not reasoning in question answering |

### 3.10.8 What the results suggest

At matched 40% savings, cross-encoder reranking has the lowest label FHR (16.4%) but misses 41.5% of paraphrases. Negation is the clearest hard-negative weakness for bi-encoders and lexical matching; cross-encoder reranking sharply reduces those hits while retaining a high paraphrase miss rate. These results support evaluating each matcher on target traffic and do not establish a universal ordering or safe threshold.

# Chapter 4 Limitations

The design limits the claims the results can support:

## 4.1 The variants come from one generator

One model (Gemma 3, 4B) generated all variants. Its wording habits may not represent user traffic; the label audit checks labels, not representativeness.

## 4.2 The label audit is small and its raters are not fully independent

The audit has ten items in each of nine strata, so it cannot estimate small error rates precisely. Two raters share a model family, and LLM raters may share biases with the judge; the consensus is diagnostic, not external ground truth.

## 4.3 The size of the corpus and dependence between queries

The 700 queries derive from 100 anchors and are clustered by source. Wilson intervals assume independence and may be too narrow; 50 examples per hard-negative type detect only large differences.

## 4.4 The kind of question

Anchors are short English factual search questions; findings may not transfer to long prompts, other languages, or the analogous applications in Section 1.2.

## 4.5 A static cache with no eviction

The static cache avoids order effects but omits insertion and eviction, which can create additional misses or stale hits in deployment.

## 4.6 An exact index

Exact search removes approximation error; production indexes may miss the true nearest neighbour (Malkov and Yashunin, 2020).

## 4.7 Local, processor-only cost

Savings count model calls avoided on local processor inference. Stored fallback latency excludes lookup time and is not net serving-time savings or an API price.

## 4.8 The judge

Phi-4-mini fails both validation gates (70/100 anchor acceptance; $\kappa = 0.416$) and is secondary only. Stored responses may themselves be wrong; judged rates depend on model build and show false rejects and accepts. Judging also adds local compute, so each pair was scored once and stored.

- Phi-4-mini decides whether a stored response is an acceptable answer to a query. It is not an independent check of factual truth. It accepts only 70 of the 100 anchors with their own stored responses and has $\kappa = 0.416$ against the consensus of the audit raters.
- The stored responses come from a 3-billion-parameter model and can be wrong for their own anchors. A judge that rejects such a response will lower the apparent quality of a cache even when the cache matched correctly.
- It rejects a substantial share of true paraphrase hits and accepts many label false hits, so the judged false-hit rate has a floor that is set by the judge. A different judge model, or a different build of the same model, could report a materially different judged rate, which makes it less reproducible than the label-based rates.
- Judging every pair for every configuration multiplies the local compute, which is why each pair is judged once and stored.

Construction-label outcomes are primary; judge measures are diagnostic until a judge passes independent validation.

## 4.9 Scores are not on a common scale

Matcher scores are not on a common scale, so shared $\tau$ comparisons are not like-for-like; matched-savings results carry the cross-matcher comparison.

## 4.10 A limited set of matching methods

Only four configurations are tested. Fine-tuned and judge-verified caches are outside scope, so the results do not rank semantic-caching approaches generally (Gill et al., 2025a, 2025b; Singh et al., 2026; Schroeder et al., 2026).

## 4.12 The matched point is chosen on the evaluation data

The 40% point is selected on the same 700 queries used to calculate errors; it is descriptive, not an estimate for new traffic.

# Chapter 5 Future Scope

The next steps follow from the limits in Chapter 4. They are plans, not results, and none of them has been carried out in this work.

## 5.1 A larger corpus and a broader audit

Increase anchors and hard negatives per type, then repeat with a second generator family. Expand each audit stratum and add independent annotators alongside model ratings to strengthen label and judge checks.

## 5.2 Deployment and domain transfer

Evaluate the same matchers on consented domain query logs. Add cache insertion, eviction, and approximate indexing to separate index error from threshold error, and add dialogue history and domain-specific traffic to test transfer.

## 5.3 Matcher and judge policies

Evaluate fine-tuned matchers and judge-verified policies on the same corpus and cost accounting. Test query-type-specific checks for negation, entities, and numbers. Validate a future judge on separately designed data or across model families before using JFH as primary.

## 5.4 Evaluation on real query traffic

Future work can evaluate the matchers on naturally occurring query logs from real applications. Such data would show how often queries repeat, how users phrase follow-up questions, and which kinds of changes cause incorrect cache hits. Before using query logs, researchers should remove personal information and follow the data provider’s privacy requirements.

## 5.5 Broader Language and Domain Coverage

The present study focuses on short English factual questions. Future work can test queries in multiple languages and domains such as healthcare, education, customer support, and software development. These settings use different terminology and have different consequences for incorrect responses.

# Chapter 6 Conclusion

This study evaluated four matchers for a static semantic cache using a controlled workload of 700 queries derived from 100 NQ-Open anchors. The workload contains 400 paraphrases, 200 hard negatives covering four types of meaning change, and 100 unrelated queries. Because matcher scores and responses were stored, the threshold analyses could be replayed consistently without rerunning model inference. This made it possible to compare cache decisions across the tested operating points while keeping the query set and stored responses fixed.

At a matched model-call savings rate of 40%, cross-encoder reranking had the lowest construction-label false-hit rate, at 16.4%. BGE cosine, MiniLM cosine, and lexical matching had rates of 26.8%, 33.6%, and 36.4%, respectively. This ordering describes the observed results on this workload; it does not establish a universal ranking. The cross-encoder's lower false-hit rate also came with a substantial cost: it missed 41.5% of paraphrases at the matched operating point. The results therefore show a practical trade-off between avoiding incorrect cache hits and capturing more paraphrases.

The type of query change also mattered. At \(\tau=0.75\), the two bi-encoders served 98% of negation queries from the cache, lexical matching served 92%, and the cross-encoder served 24%. Since a negation change can reverse what a question asks, the high bi-encoder hit rates indicate a specific weakness of similarity-based matching on this hard-negative type. Negation queries also had a Jaccard similarity of 0.74, showing why strong lexical overlap can coexist with a changed meaning. P2 and P3 paraphrases had nearly equal lexical distance, which suggests that the lexical matcher does not reliably separate those two paraphrase levels using wording distance alone.

The response judge did not meet the prespecified validation gates. It accepted 70 of the 100 stored anchor responses and agreed with the audit consensus on 71.1% of the 90 audited items, with Cohen's \(\kappa=0.416\). The judge also rejected many true paraphrase hits and accepted many hits that were false under the construction labels. For these reasons, judge-based false-hit rates are treated as secondary diagnostics, while the primary comparison uses the labels defining the query–anchor relationships. This choice limits claims about the factual

quality of every cached response: construction labels measure whether the cache selected the intended anchor, not whether its response is independently factually correct.

Taken together, the findings support evaluating semantic caches by both their savings and their errors, with results broken down by the kinds of query changes they encounter. A single threshold value cannot be assumed to have the same meaning across matchers with different score scales, and a shared cutoff can conceal meaningful differences in their decisions. Matched-savings comparisons provide a more interpretable view of the observed cost–correctness trade-off, while confidence-interval overlap means that some apparent differences should not be treated as established.

The study has several boundaries. Its corpus is generated and modest in size, and it focuses on short, single-turn English questions. The cache is static, and the experiments use local processor inference. The results therefore describe the tested workload and setup; they do not establish a deployment-safe threshold or guarantee that one matcher will perform best on other query populations. Future evaluations should test naturally occurring traffic, broader domains and languages, multi-turn context, and cache updates, while independently checking the quality of reused answers.

## Judge prompt

The prompt below was applied unchanged at temperature 0 to every pair of a query, the matched anchor and the stored response, and to each anchor with its own stored response.

> Evaluate whether the cached response answers the incoming question correctly. Judge meaning and factual adequacy, not wording. Use the reference answers when present, but assess independently if they are absent. Return JSON with one key verdict whose value is exactly "correct_hit" or "false_hit".
>
> Incoming question: {query_text}
>
> Matched anchor: {anchor_question}
>
> Cached response: {anchor_answer}
>
> Reference answers, if available: {gold_answers}

The braces mark the places where the pair's text is inserted.

## References


1. Bang, F. (2023) 'GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings', Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), Singapore, pp. 212–218.
2. Brown, L.D., Cai, T.T. and DasGupta, A. (2001) 'Interval estimation for a binomial proportion', Statistical Science, Vol. 16, No. 2, pp. 101–133.
3. Chaudhuri, S., Ganti, V. and Kaushik, R. (2006) 'A primitive operator for similarity joins in data cleaning', Proceedings of the 22nd International Conference on Data Engineering (ICDE).
4. Chen, L., Zaharia, M. and Zou, J. (2023) 'FrugalGPT: How to use large language models while reducing cost and improving performance', arXiv:2305.05176.
5. Cohen, J. (1960) 'A coefficient of agreement for nominal scales', Educational and Psychological Measurement, Vol. 20, No. 1, pp. 37–46.
6. Cross-Encoder (2026) 'cross-encoder/quora-distilroberta-base: Cross-Encoder for Quora Duplicate Questions Detection', Hugging Face model card, accessed 30 October 2026, https://huggingface.co/cross-encoder/quora-distilroberta-base.
7. Gao, T., Yao, X. and Chen, D. (2021) 'SimCSE: Simple contrastive learning of sentence embeddings', Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6894–6910.
8. Gardner, M. et al. (2020) 'Evaluating models' local decision boundaries via contrast sets', Findings of the Association for Computational Linguistics: EMNLP 2020.
9. Gemma Team (2025) 'Gemma 3 technical report', arXiv:2503.19786.
10. Gill, W., Elidrisi, M., Kalapatapu, P., Ahmed, A., Anwar, A. and Gulzar, M.A. (2025a) 'MeanCache: User-centric semantic caching for LLM web services', Proceedings of the IEEE 39th International Parallel and Distributed Processing Symposium (IPDPS); arXiv:2403.02694.
11. Gill, W. et al. (2025b) 'Advancing semantic caching for LLMs with domain-specific embeddings and synthetic data', arXiv:2504.02268.
12. Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A. and Zhong, L. (2023) 'Prompt Cache: Modular attention reuse for low-latency inference', arXiv:2311.04934.
13. Jin, C. et al. (2024) 'RAGCache: Efficient knowledge caching for retrieval-augmented generation', arXiv:2404.12457.
14. Johnson, J., Douze, M. and Jégou, H. (2021) 'Billion-scale similarity search with GPUs', IEEE Transactions on Big Data, Vol. 7, No. 3, pp. 535–547.
15. Karpukhin, V. et al. (2020) 'Dense passage retrieval for open-domain question answering', Proceedings of EMNLP, pp. 6769–6781.
16. Kwiatkowski, T. et al. (2019) 'Natural Questions: A benchmark for question answering research', Transactions of the Association for Computational Linguistics, Vol. 7, pp. 453–466.
17. Landis, J.R. and Koch, G.G. (1977) 'The measurement of observer agreement for categorical data', Biometrics, Vol. 33, No. 1, pp. 159–174.
18. Lee, K., Chang, M.-W. and Toutanova, K. (2019) 'Latent retrieval for weakly supervised open domain question answering', Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 6086–6096.
19. Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C. and Carlini, N. (2022) 'Deduplicating training data makes language models better', Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8424–8445.

20. Lewis, P. et al. (2020) 'Retrieval-augmented generation for knowledge-intensive NLP tasks', Advances in Neural Information Processing Systems, Vol. 33, pp. 9459–9474.

21. Lhoest, Q. et al. (2021) 'Datasets: A community library for natural language processing', Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 175–184.

22. Li, J., Xu, C., Wang, F., von Riedemann, I.M., Zhang, C. and Liu, J. (2024) 'SCALM: Towards semantic caching for automated chat services with large language models', arXiv:2406.00025.

23. Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R. and Zhu, C. (2023) 'G-Eval: NLG evaluation using GPT-4 with better human alignment', Proceedings of EMNLP, Singapore, pp. 2511–2522.

24. Malkov, Y.A. and Yashunin, D.A. (2020) 'Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs', IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 42, No. 4, pp. 824–836.

25. Meta AI (2024) "Llama 3.2 model card", Meta Llama, https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md (accessed 1 November 2026).

26. Microsoft (2025) 'Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs', arXiv:2503.01743.

27. Muennighoff, N., Tazi, N., Magne, L. and Reimers, N. (2023) 'MTEB: Massive text embedding benchmark', Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp. 2014–2037.

28. Nogueira, R. and Cho, K. (2019) 'Passage re-ranking with BERT', arXiv:1901.04085.

29. Panickssery, A., Bowman, S.R. and Feng, S. (2024) 'LLM evaluators recognize and favor their own generations', Advances in Neural Information Processing Systems, Vol. 37.

30. Regmi, S. and Pun, C.P. (2024) 'GPT Semantic Cache: Reducing LLM costs and latency via semantic embedding caching', arXiv:2411.05276.

31. Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence embeddings using Siamese BERT-networks', Proceedings of EMNLP-IJCNLP, Hong Kong, pp. 3982–3992.

32. Ribeiro, M.T., Wu, T., Guestrin, C. and Singh, S. (2020) 'Beyond accuracy: Behavioral testing of NLP models with CheckList', Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics.

33. Schroeder, L.G. et al. (2026) 'vCache: Verified semantic prompt caching', International Conference on Learning Representations (ICLR); arXiv:2502.03771.

34. Singh, A.K., Wang, H., Attaluri, S., Chiam, T. and Zhu, W. (2026) 'Asynchronous verified semantic caching for tiered LLM architectures', arXiv:2602.13165.

35. Truong, T.H., Baldwin, T., Verspoor, K. and Cohn, T. (2023) 'Language models are not naysayers: An analysis of language models on negation benchmarks', Proceedings of *SEM 2023.

36. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N. and Zhou, M. (2020) 'MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers', Advances in Neural Information Processing Systems, Vol. 33, pp. 5776–5788.

37. Wilson, E.B. (1927) 'Probable inference, the law of succession, and statistical inference', Journal of the American Statistical Association, Vol. 22, No. 158, pp. 209–212.

38. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D. and Nie, J.-Y. (2023) 'C-Pack: Packed resources for general Chinese embeddings', arXiv:2309.07597.

39. Yan, J., Ni, W., Chen, L., Lin, X., Cheng, P., Qin, Z. and Ren, K. (2025) 'ContextCache: Context-aware semantic cache for multi-turn queries in large language models', Proceedings of the VLDB Endowment, Vol. 18, No. 12, pp. 5391–5394.

40. Yu, C., Wang, T., Shao, Z. and Jiang, S. (2025) 'SmartCache: Context-aware semantic cache for efficient multi-turn LLM inference', Advances in Neural Information Processing Systems (NeurIPS).

41. Zhang, Y., Baldridge, J. and He, L. (2019) 'PAWS: Paraphrase adversaries from word scrambling', Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), pp. 1298–1308.

42. Zheng, L. et al. (2023) 'Judging LLM-as-a-judge with MT-Bench and Chatbot Arena', Advances in Neural Information Processing Systems, Vol. 36.